Yegor Polyakov

Buenos Aires, Argentina · +54 11 2796-5334
egor-polyakov.vercel.app · egor.pol9kov@gmail.com

Independent · Imperia OS · 2025 – present

Building an AI assistant to do everything its user wants — and the platform that goal requires.

The same infrastructure builds the platform and delivers it to its users.

The platform is written from inside itself. Agents propose a change, author it, verify it, and land it behind a gate; deployment walks dev, standby, and production with no hand on it. Most of the code is written by the system now. What stays with me is direction, and the gate.

Two kinds of memory back the development, and both reach the agent the same two ways: pulled into its prompt automatically by embedding match, or fetched on demand through MCP tools.

The first is a knowledge graph that follows the development flow: business use cases come first; tests are written from them; code implements them; and all three are automatically indexed into the graph and embedded.

The second is a thought system: a hierarchy that composes itself, where every new thought is summarized into the layer above, recursively. The same module runs the business agents.

Another way of working with memory: state. A stateful agent is a state machine — the algorithm controls the flow, and each step is atomic: an LLM call or a deterministic operation. The algorithm doesn't hallucinate. Stateless agents are the rest: one call, no machine.

People and agents are members of the same rooms. Membership is privacy and addressing at once — another agent, another contact, a private side channel between two of them is one more room, not one more concept.

One agent stands between that mesh and the person. Everything the others say passes through it, and only what changes what the person would do reaches them.

A promise is an object, not a sentence. When a turn commits to something and nothing durable backs it, the debt is written down and the agent is woken on it later. Going silent on a promise is a state the system can see.

Agents are one of three abstractions the system reduces to. Blocks are the second — everything addressable shares one shape and one embedding. Protocols are the third — everything that crosses a boundary. Payment included.

Full-Stack Engineer · Independent · 2022 – 2025

A cross-platform mobile platform on Flutter and AWS. Clean Architecture from the first commit, cloud infrastructure as code in AWS CDK.

Dart and Flutter on the frontend — feature-based modules, event-driven reactive layout, Riverpod for state. AWS Lambda in Go on the backend — DynamoDB with composite keys, S3 for media, Amplify and Cognito for auth.

Blockchain Developer · IBM · 2020 – 2022

Built a full testing environment and CI/CD pipeline for Hyperledger Fabric — feature delivery speed doubled. Refactored the project structure, cutting the codebase by 30%. Led the CouchDB→LevelDB migration that improved data-operation performance by 40%. Delivered a cross-channel encryption solution with zero defects, despite the absence of formal requirements.

Blockchain Developer · Insolar · 2019 – 2020

Designed the data architecture for a blockchain-based wallet, initiating the process and guiding a newly onboarded system architect through the key design decisions.

Blockchain Developer · Sber · 2018 – 2019

A custom CRUD-style abstraction layer over Hyperledger Fabric, self-initiated and built after hours. New-feature implementation dropped from around eight hours to fifteen minutes, simplifying composite-key data operations.

St. Petersburg State University, Mat-Mekh · 2012 – 2017

Mathematics and Mechanics faculty. Arrived from LNMO, the Saint Petersburg laboratory for continuous mathematical education. Olympiad medals along the way — mathematics at ICYS and the City Olympiad, computer science at the Baltic Scientific and Engineering Competition. Worked through university in parallel — Java, Android, Delphi, and embedded C++ for spectrograph instrumentation at a materials research institute.